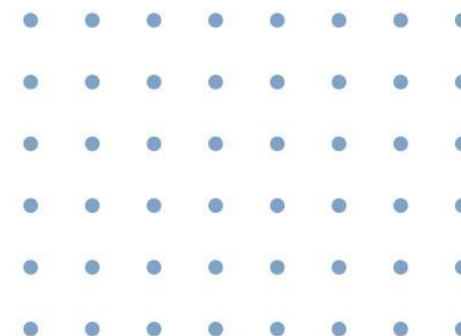
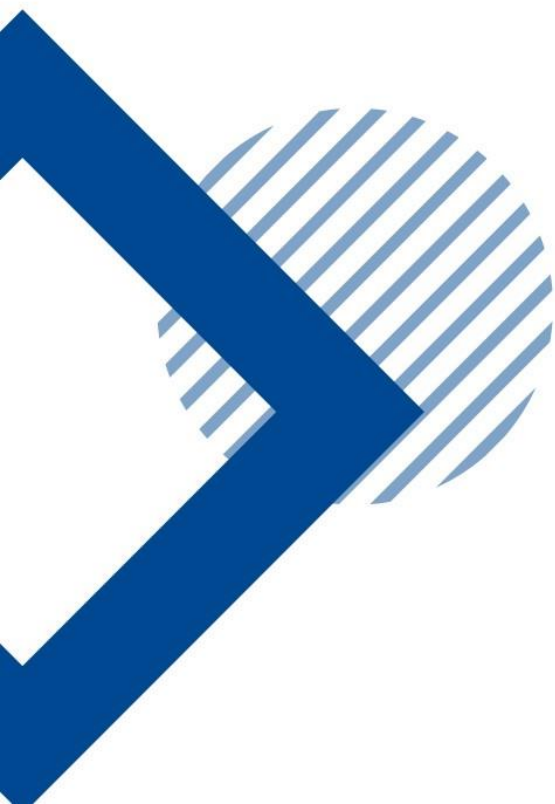


Лекция №6

Взаимоблокировка



Основные понятия

Взаимоблокировка в группе процессов возникает в том случае, если каждый процесс из этой группы ожидает события, наступление которого зависит исключительно от другого процесса из этой же группы.

Выгружаемый ресурс – ресурс, который безболезненно можно забрать у процесса (например: память).

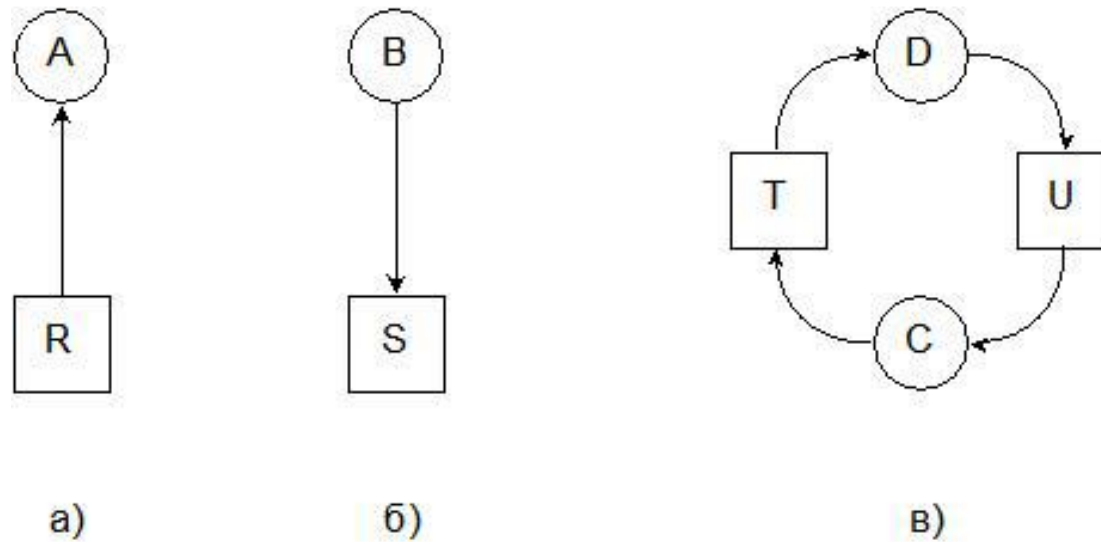
Невыгружаемый ресурс – ресурс, который нельзя забрать у процесса без потери данных (например: принтер).

Проблема взаимоблокировок процессов возникает при борьбе за невыгружаемые ресурсы.

Условия необходимые для возникновения взаимоблокировки:

1. Условие взаимного исключения – в какой-то момент времени, ресурс занят только одним процессом или свободен.
2. Условие удержания и ожидания – процесс, удерживающий ресурс может запрашивать новые ресурсы.
3. Условие отсутствия принудительной выгрузки ресурса.
4. Условие циклического ожидания – должна существовать круговая последовательность из процессов, каждый, из которого ждет доступа к ресурсу, удерживаемому следующим членом последовательности.

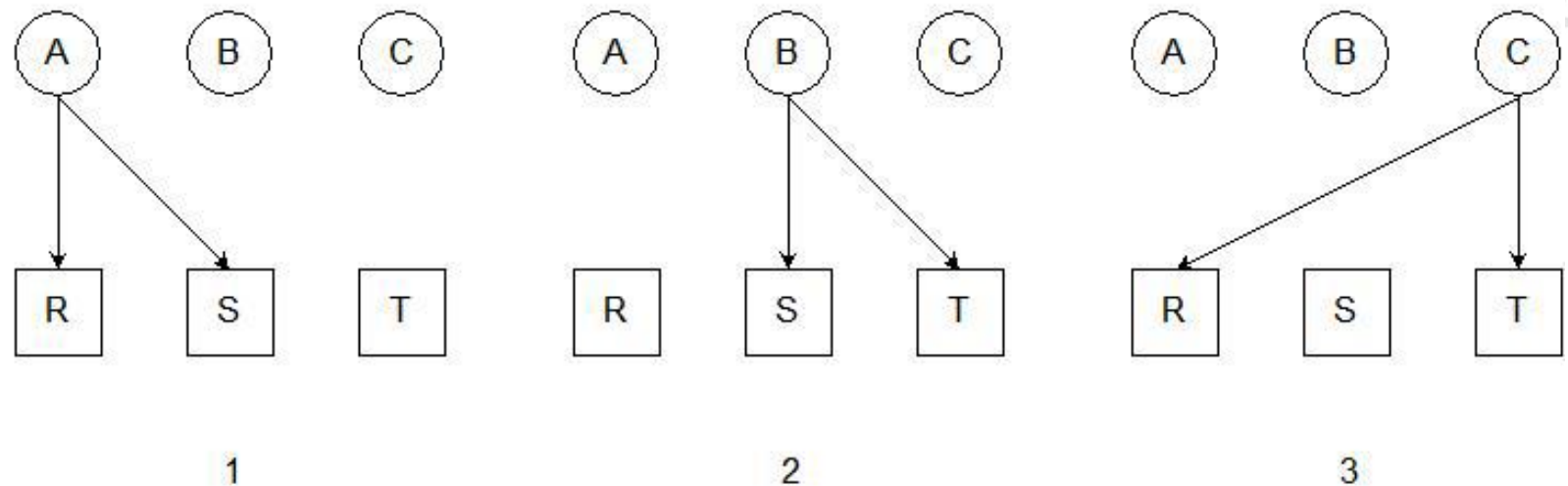
Моделирование взаимоблокировок



Графы распределения ресурсов:
ресурс занят а); запрос ресурса б); взаимоблокировка в)

Последовательное выполнение процессов

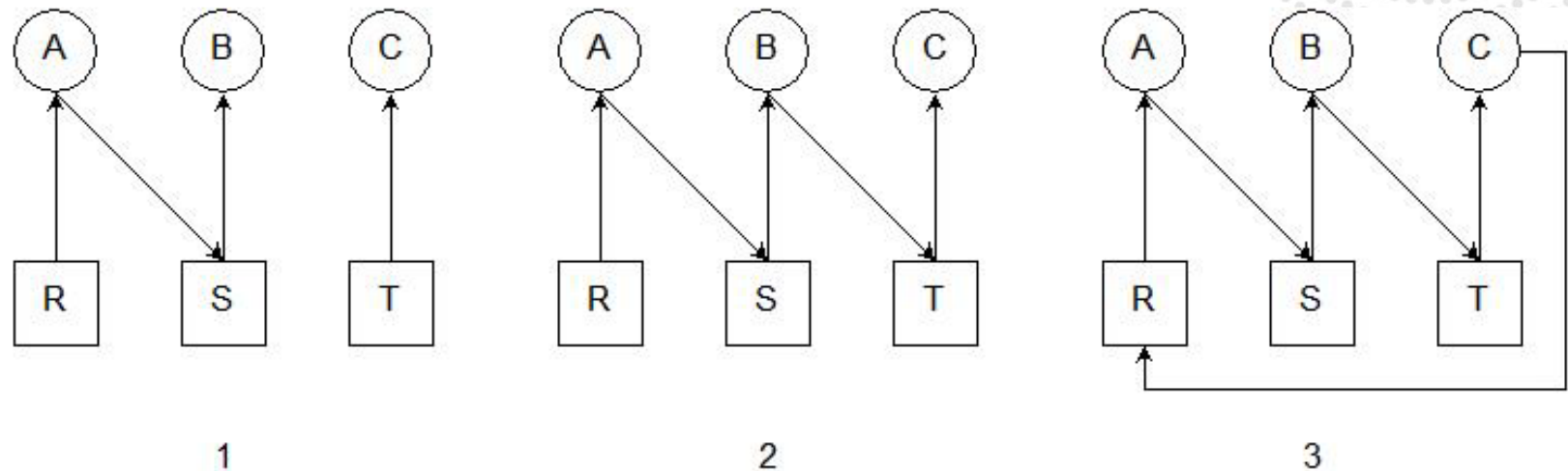
1. A запрашивает R
A запрашивает S
Освободить R
Освободить S
 2. B запрашивает S
B запрашивает T
Освободить S
Освободить T
 3. C запрашивает T
C запрашивает R
Освободить T
Освободить R
- Взаимоблокировки нет



Пример последовательного выполнения процессов.
Взаимоблокировки нет.

Возникновение взаимоблокировки

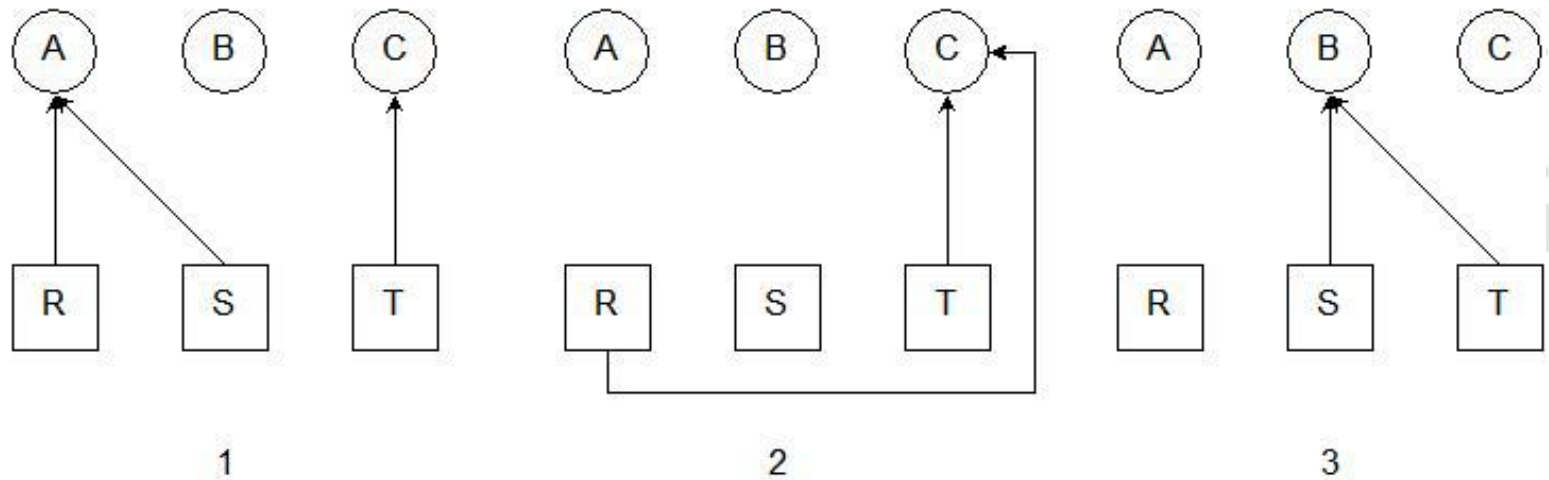
1. A выделен R
A запрашивает S
 2. B выделен S
B запрашивает T
 3. C выделен T
C запрашивает R
- Взаимоблокировка



Пример возникновения взаимоблокировки

Предупреждение взаимоблокировки

1. A выделен R
A запрашивает S
A выделен S
C выделен T
 2. C запрашивает R
Освободить R
Освободить S
C выделен R
 3. B запрашивает T
Освободить T
Освободить R
B выделен T
B запрашивает S
B выделен S
- Взаимоблокировки нет



Пример предупреждения взаимоблокировки

Методы борьбы с взаимоблокировками

Четыре стратегии для борьбы с взаимными блокировками:

1. Игнорирование проблемы.
2. Обнаружение и восстановление.
3. Динамическое уклонение от взаимоблокировок за счет тщательного распределения ресурсов.
4. Предотвращение взаимоблокировок за счет структурного подавления одного из четырех условий необходимых для их возникновения.

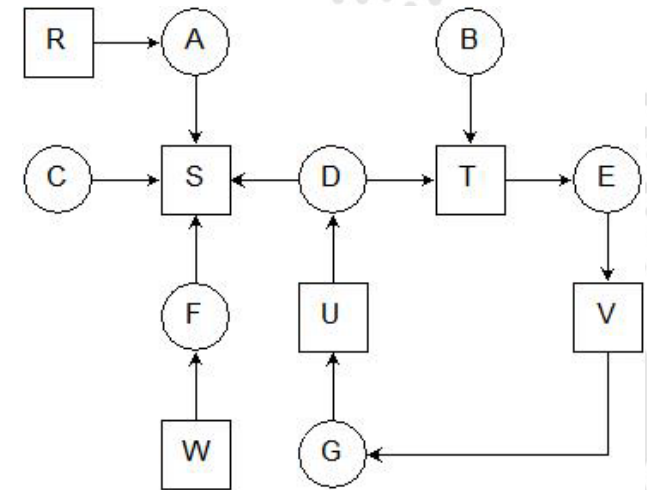
Игнорирование проблемы

Страусиный алгоритм – если вероятность взаимоблокировки очень мала, то ею легче пренебречь, т. к. код исключения может усложнить ОС и привести к возникновению ошибок. Также многие взаимоблокировки тяжело обнаружить. Этот алгоритм используется во многих современных ОС (UNIX-like, семейство Windows).

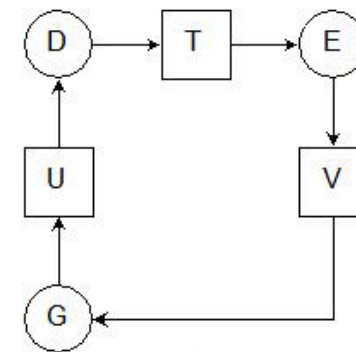
Обнаружение взаимоблокировки при наличии одного ресурса каждого типа

Для каждого узла N в графе выполняется пять шагов:

1. Очищение списка L (список обработанных узлов).
2. Текущий узел добавляется в конец списка L. Если данный узел уже содержится в списке, то граф содержит цикл, алгоритм прекращает работу.
3. Определение для текущего узла количества отходящих от него непомеченных ребер. Если такие ребра существуют, переход к шагу 4, иначе переход к шагу 5.
4. Выбор и пометка отходящего от узла ребра, определение нового текущего узла, переход к шагу 2.
5. Если текущий узел является первоначальным узлом: граф не содержит циклов, алгоритм прекращает работу. Иначе предыдущий узел становится текущим, переход к шагу 2.



а)



б)

Граф ресурсов а);
извлеченный из него цикл б).

Обнаружение взаимоблокировки при наличии нескольких ресурсов каждого типа

1. Поиск непомеченного процесса P_i , для которого i -я строка матрицы R меньше или равна A .
2. Если такой процесс найден, прибавляем к A i -ую строку матрицы C . Установка метки на i -ый процесс, переход к шагу 1.
3. Если такого процесса нет, алгоритм прекращает работу.

По окончании работы алгоритма все непомеченные процессы, считаются участвующими во взаимоблокировке.
Пример работы алгоритма:

1. Третий процесс может получить желаемые ресурсы, т.к. $R_3 = A$.
2. Третий процесс освобождает ресурсы. $A = (2, 2, 2, 0)$. Процесс маркируется как выполненный.
3. Выполняется процесс 2. По окончании $A = (4, 2, 2, 1)$.
4. Выполняется первый процесс.

Существующие ресурсы
($E_1, E_2, E_3, \dots, E_m$)

Доступные ресурсы
($A_1, A_2, A_3, \dots, A_m$)

Матрица текущего
распределения

Матрица запроса

C_{11}	C_{12}	C_{13}	...	C_{1m}
C_{21}	C_{22}	C_{23}	...	C_{2m}
...				
C_{n1}	C_{n2}	C_{n3}	...	C_{nm}

R_{11}	R_{12}	R_{13}	...	R_{1m}
R_{21}	R_{22}	R_{23}	...	R_{2m}
...				
R_{n1}	R_{n2}	R_{n3}	...	R_{nm}

Строка n предоставлена
процессу n

Строка 2 нужна процессу 2

а)

$E = (4, 2, 3, 1)$

$A = (2, 1, 0, 0)$

$$C = \begin{vmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{vmatrix}$$

$$R = \begin{vmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{vmatrix}$$

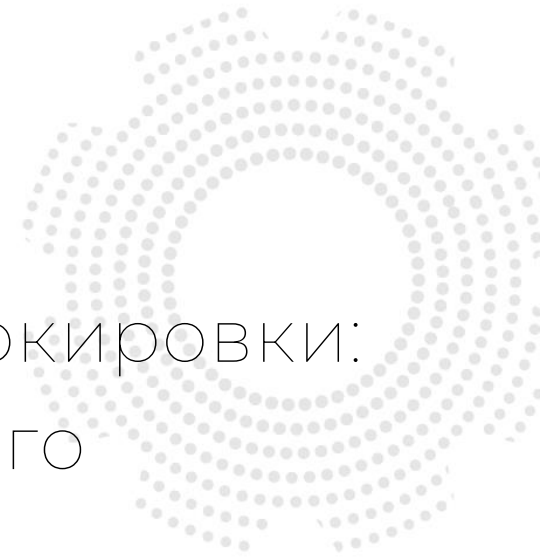
б)

Структуры данных, необходимые
для работы алгоритма а);
пример работы алгоритма б).

Выход из взаимоблокировки

Методы выхода из состояния взаимоблокировки:

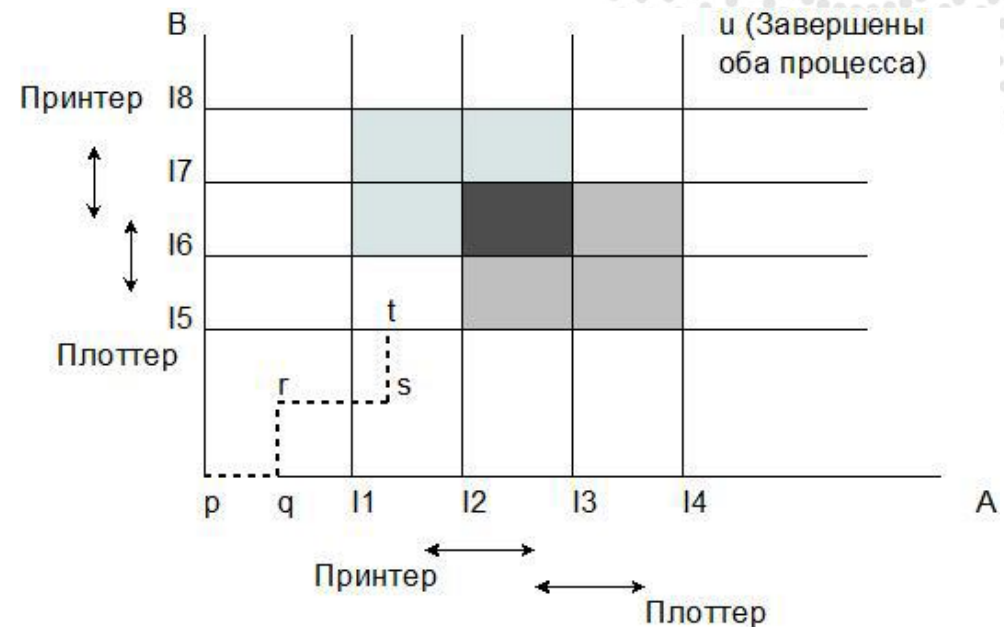
1. Восстановление за счет приоритетного владения ресурсом.
2. Восстановление путем отката.
3. Восстановление путем уничтожения процессов.



Динамическое уклонение от взаимоблокировок за счет тщательного распределения ресурсов

Траектории ресурсов:

1. I1 – запрос принтера процессом А.
2. I2 – запрос плоттера процессом А.
3. I3 – освобождение принтера процессом А.
4. I4 – освобождение плоттера процессом А.
5. I5 – запрос плоттера процессом В.
6. I6 – запрос принтера процессом В.
7. I7 – освобождение плоттера процессом В.
8. I8 – освобождение принтера процессом В.



Траектории ресурсов
двух процессов

Безопасные и небезопасные состояния

	3	Н
А	3	9
В	2	4
С	2	7
Свободно: 3		

	3	Н
А	3	9
В	4	4
С	2	7
Свободно: 1		

	3	Н
А	3	9
В	-	-
С	2	7
Свободно: 5		

	3	Н
А	3	9
В	-	-
С	7	7
Свободно: 0		

	3	Н
А	3	9
В	-	-
С	-	-
Свободно: 7		

а)

	3	Н
А	3	9
В	2	4
С	2	7
Свободно: 3		

	3	Н
А	4	9
В	2	4
С	2	7
Свободно: 2		

	3	Н
А	4	9
В	4	4
С	2	7
Свободно: 0		

	3	Н
А	4	9
В	-	-
С	2	7
Свободно: 4		

б)

3 - Занято ресурсов
Н - Необходимо ресурсов

Пример безопасного состояния а);
пример небезопасного состояния б).

Алгоритм банкира при наличии одного ресурса каждого типа

1. Банкиру поступает запрос от клиента на получение кредита.
2. Банкир проверяет, приводит ли этот запрос к небезопасному состоянию.
3. Зависимости от типа ситуации банкир дает или отказывает в кредите.

	З	Н
А	0	6
В	0	5
С	0	4
Д	0	7

Свободно: 10

а)

	З	Н
А	1	6
В	1	5
С	2	4
Д	3	7

Свободно: 2

б)

	З	Н
А	1	6
В	2	5
С	2	4
Д	4	7

Свободно: 1

в)

Состояние распределения ресурсов:
безопасное а); безопасное б);
небезопасное в).

Алгоритм банкира при наличии нескольких ресурсов каждого типа

1. Поиск процесса в матрице R, потребности которого меньше или равны вектору A. Если искомая строка не существует, то система окажется в состоянии взаимной блокировки. Прекращение работы алгоритма.
2. Предполагается, что процесс завершился и освободил занимаемые им ресурсы. Процесс помечается как завершённый, освободившиеся ресурсы добавляются в вектор A.
3. Повторение шагов 1 и 2 пока все процессы не будут помечены как завершённые (безопасное состояние), либо не останется процессов, чьи запросы могут быть удовлетворены (небезопасное состояние).

$$E = (6, 3, 4, 2)$$

$$P = (5, 3, 2, 2)$$

$$A = (1, 0, 2, 0)$$

		E1	E2	E3	E4
C =	A	3	0	1	1
	B	0	1	0	0
	C	1	1	1	0
	D	1	1	0	1
	E	0	0	0	0

Матрица текущего распределения

		E1	E2	E3	E4
R =	A	1	1	0	0
	B	0	1	1	2
	C	3	1	0	0
	D	0	0	1	0
	E	2	1	1	0

Матрица запроса

Пример работы алгоритма банкира для несколько видов ресурсов.

Предотвращение взаимоблокировки

Методы предотвращения взаимоблокировок за счет структурного подавления одного из четырех условий необходимых для их возникновения:

1. Предотвращение условия взаимного исключения. Минимизация количества процессов борющихся за ресурсы. Например, с помощью очереди печати, когда только демон/сервис принтера работает с ним.
2. Предотвращение условия удержания и ожидания. Один из способов достижения этой цели – запрос процессом всех необходимых ресурсов до начала работы. В случае недоступности хотя бы одного ресурса процессу ничего не предоставляется.
3. Предотвращение условия отсутствия принудительной выгрузки ресурса.
4. Предотвращение условия циклического ожидания.
Способы предотвращения:
 - процесс должен освободить занятый ресурс, прежде чем занять новый;
 - нумерация всех ресурсов. Процесс может запрашивать ресурсы только в порядке возрастания их номеров.